CYLIXLAB · STUDY GUIDE

Module 03

Natural Language Processing & Transformer Models

Theory | Tokenization · Embeddings · Self-Attention · Multi-Head Attention · BERT · GPT

What you will master

Section	Topic	You should be able to…
1	Tokenization	Explain word, character and subword tokenizers and run BPE merges by hand
2	Embeddings	Describe static vs contextual embeddings, cosine similarity and positional encodings
3	Self-Attention	Compute scaled dot-product attention and explain Q, K, V, masking and the Transformer block
4	Multi-Head Attention	Explain why multiple heads help and count attention and layer parameters
5	BERT	Describe the encoder-only design, masked language modelling and fine-tuning
6	GPT	Describe the decoder-only design, causal masking, generation and decoding strategies
7	Revision Kit	Compare BERT, GPT and T5, use the formula sheet and avoid classic mistakes

KEY IDEA

The big idea: text is split into **tokens**, each token becomes a vector (**embedding**), and **self-attention** lets every token build a context-aware representation by looking at all other tokens at once. Stack these blocks and you get a **Transformer**. Keep only the encoder and train it to fill in masked words and you get **BERT**; keep only the decoder and train it to predict the next token and you get **GPT**.

Era	Approach	Limitation that motivated the next step
Before ~2013	Bag-of-words, TF-IDF + linear models	No word order, no meaning of similarity; huge sparse vectors
2013–2016	Word2Vec / GloVe embeddings + RNN/LSTM	One vector per word (no context); sequential, slow to train
2014–2017	Seq2seq + attention	Still built on recurrent networks
2017	**Transformer** (“Attention Is All You Need”)	Parallel, long-range; the foundation of everything below
2018 →	**BERT** (encoder), **GPT** (decoder), T5 (encoder–decoder)	Pre-train on huge unlabelled text, then adapt to many tasks

1 Tokenization

Neural networks work on numbers, not text. **Tokenization** converts a string into a sequence of **tokens** from a fixed vocabulary, and each token is mapped to an integer **ID**. The choice of tokenizer affects vocabulary size, sequence length, how rare words are handled and the cost of running a model.

unhappiness ⟶ [un, happi, ness] ⟶ [403, 17821, 1394]

Level	Example tokens for “unhappiness”	Pros	Cons
Word	unhappiness	Short sequences, intuitive	Huge vocabulary; **out-of-vocabulary (OOV)** words; no sharing between “happy” and “unhappy”
Character	u n h a p p i …	Tiny vocabulary; no OOV	Very long sequences; harder to learn meaning
Subword	un + happi + ness	Compact vocabulary, no OOV, shares morphemes	Splits can look odd; depends on training corpus

1.1 Byte-Pair Encoding (BPE)

BPE starts from characters and repeatedly **merges the most frequent adjacent pair** into a new token until the vocabulary reaches a target size (typically 30,000–100,000). Frequent words end up as single tokens; rare words are split into pieces.

WORKED EXAMPLE
Corpus (word : frequency): *low* : 5, *lower* : 2, *newest* : 6, *widest* : 3. Start with characters plus an end-of-word marker </w>.
Pair counts: (e, s) = 6 + 3 = **9**, (s, t) = 9, (t, </w>) = 9, (w, e) = 2 + 6 = 8, (l, o) = 7, (o, w) = 7.
Merge 1: e + s → **es** (9). **Merge 2:** es + t → **est** (9). **Merge 3:** est + </w> → **est</w>** (9). Next come l + o → lo and lo + w → low (7). The new word “lowest” can now be tokenized as *low* + *est</w>* even though it never appeared in training.

Algorithm	Used by	Idea
BPE	GPT family (byte-level BPE), RoBERTa	Merge the most **frequent** pair; byte-level versions can encode any text with no OOV
WordPiece	BERT	Merge the pair that most increases training-data **likelihood**; continuation pieces are marked with ##
Unigram / SentencePiece	T5, many multilingual models	Start large and prune tokens that least hurt likelihood; works directly on raw text, including languages without spaces

1.2 Special tokens and practical effects

- **[CLS]** (summary token for classification), **[SEP]** (separates sentences), **[MASK]** (hidden word), **[PAD]** (padding to equal length), **[UNK]** (unknown). GPT-style models use markers such as an end-of-text token.
- Models have a **maximum sequence length** (e.g. 512 tokens for BERT); longer inputs are truncated or chunked. API cost and speed scale with the **token count**, not the number of words (English averages roughly 1.3 tokens per word).
- Tokenizers are **case-, language- and domain-dependent**: numbers, code, emojis and non-English text often split into many tokens, which hurts efficiency and sometimes accuracy.
- A model must be used with the **exact tokenizer** it was trained with; a different vocabulary produces meaningless IDs.

COMMON PITFALL
Token ≠ word. Counting words to estimate context-window usage, or assuming “one character = one token”, leads to truncated inputs and surprise costs. Always count tokens with the model’s own tokenizer.

2 Embeddings

A one-hot vector for a 30,000-word vocabulary is huge, sparse and treats every pair of words as equally different. An **embedding** maps each token ID to a **dense, learned vector** (typically 128–4096 dimensions) so that similar tokens have nearby vectors.

$$\mathbf{e}_i = E[i], \qquad E \in \mathbb{R}^{V \times d} \ (V = \text{vocabulary size},\ d = \text{embedding size})$$

The embedding layer is a **lookup table** whose rows are trained by backpropagation along with the rest of the model. It has **V·d parameters** (BERT-base: 30,522 × 768 ≈ 23.4 million).

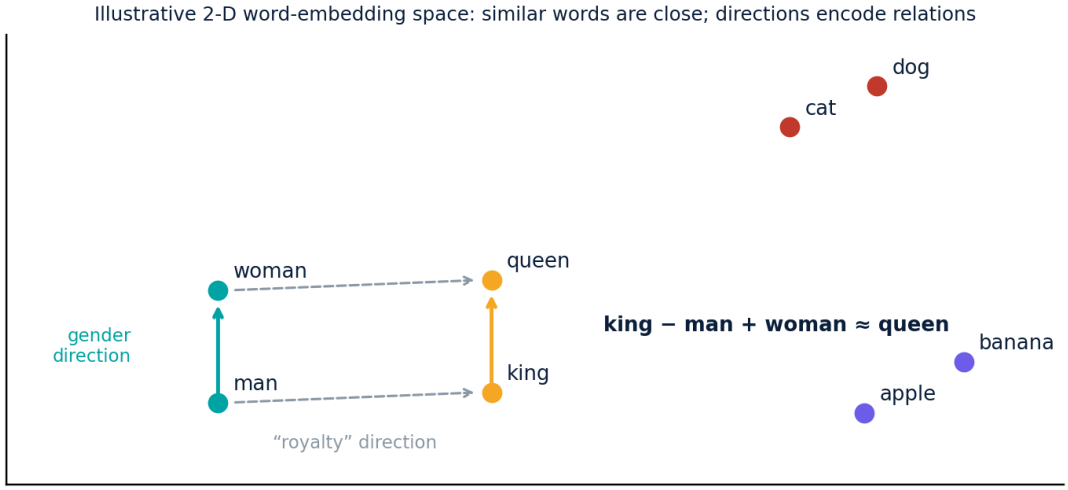


Figure 1: Directions in embedding space can capture relations such as gender or royalty (illustrative).

2.1 Static embeddings: Word2Vec and GloVe

Word2Vec learns vectors by predicting a word from its neighbours (CBOW) or neighbours from a word (skip-gram); **GloVe** factorizes global co-occurrence statistics. Both give **one fixed vector per word**. Limitation: **polysemy**. “bank” in *river bank* and *bank loan* receives the same vector.

2.2 Contextual embeddings

In ELMo, BERT and GPT the vector for a token depends on the **whole sentence**: the output of each Transformer layer is a context-specific representation, so the two uses of “bank” get different vectors. The input embedding is only the starting point; the layers refine it.

2.3 Measuring similarity

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|} \in [-1, 1]$$

WORKED EXAMPLE
u = [1, 2, 2], v = [2, 1, 2]: u·v = 2 + 2 + 4 = 8; ‖u‖ = √9 = 3; ‖v‖ = 3. cos = 8/(3·3) = **0.889** (very similar). Orthogonal vectors give 0; opposite vectors give −1. Cosine ignores vector length, which is why it is the default for semantic search.

2.4 Positional encoding: injecting word order

Self-attention treats its input as a **set**: shuffling the tokens would give shuffled but otherwise identical outputs. To use word order, a **position vector** is added to each token embedding.

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- **Sinusoidal** (original Transformer): fixed, can extrapolate to unseen lengths. **Learned** position embeddings (BERT, GPT-2): a trainable table limited to the maximum length.
- **Relative / rotary (RoPE)** schemes encode distance between tokens inside the attention computation and are common in recent large language models.
- **Sentence embeddings** are produced by pooling token vectors (mean pooling or [CLS]) and are used for search, clustering and retrieval-augmented generation (RAG).

3 Self-Attention

Consider “The animal didn’t cross the street because **it** was tired.” To understand “it” the model must link it to “animal”. **Self-attention** lets every token compute a weighted average of all tokens’ information, with weights learned from content, so the link can be formed in **one step** regardless of distance. (An RNN would need to carry the information through every intermediate step.)

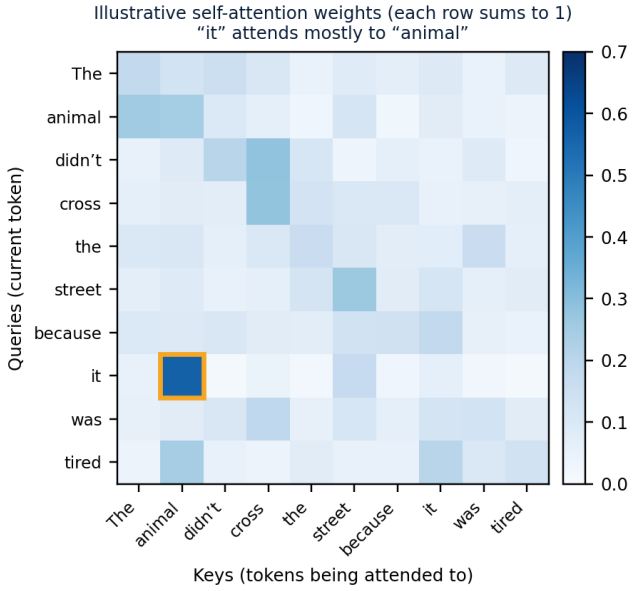


Figure 2: Attention weights for one head (illustrative). The row for “it” puts most weight on “animal”.

3.1 Queries, keys and values

Each token vector x_i is projected by three learned matrices into a **query** q_i (“what am I looking for?”), a **key** k_i (“what do I contain?”) and a **value** v_i (“what information do I pass on?”). Similarity between a query and each key decides how much of each value is mixed into the output.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad \text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

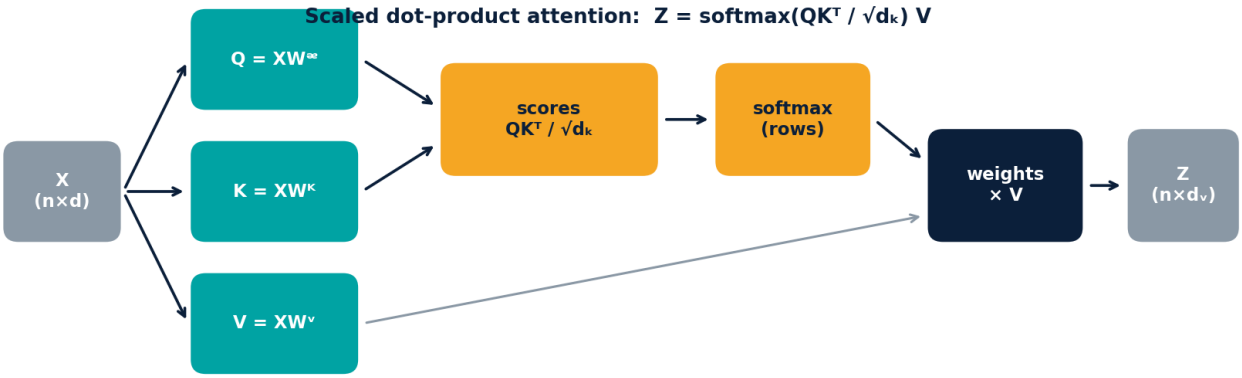


Figure 3: Data flow of scaled dot-product attention.

WORKED EXAMPLE
One query q = [1, 0] attends to two tokens: k_1 = [1, 0], k_2 = [0, 1] with values v_1 = [1, 2], v_2 = [3, 4]; d_k = 2. Scores: q·k_1 = 1, q·k_2 = 0, scaled by √2 → [0.707, 0]. Softmax: $e^{0.707}$ = 2.028, e^0 = 1, so weights = **[0.670, 0.330]**.
Output = 0.670·[1, 2] + 0.330·[3, 4] = **[1.66, 2.66]**: a blend leaning toward the better-matching token.

3.2 Why divide by $\sqrt{d_k}$?

If the components of q and k have unit variance, the dot product q·k has variance $\mathbf{d_k}$. For large d_k the scores become large in magnitude, pushing softmax into saturated regions where one weight ≈ 1 and the gradients ≈ 0. Dividing by $\sqrt{d_k}$ restores unit variance and keeps training stable (for d_k = 64 the divisor is 8).

3.3 Shapes, cost and masking

Object	Shape (n tokens)	Comment
X	$n \times d_{model}$	Input token representations
W_Q, W_K, W_V	$d_{model} \times d_k$ (or d_v)	Learned projections
Score matrix QK^T	**n × n**	Every token vs every token
Output Z	$n \times d_v$	Context-aware representations

- **Complexity:** time and memory grow as **O(n²)** in sequence length, which limits context length (an RNN is O(n) but strictly sequential). Efficient variants (FlashAttention, sparse or sliding-window attention) reduce the cost.
- **Padding mask:** scores for [PAD] tokens are set to −∞ before softmax so they receive zero weight.
- **Causal mask:** scores for *future* positions are set to −∞, so token t can only attend to tokens ≤ t (used in GPT; Section 6).
- **Self-attention vs cross-attention:** in cross-attention (encoder–decoder models) the queries come from the decoder and the keys/values from the encoder.

3.4 The Transformer block

A Transformer layer wraps attention with a position-wise **feed-forward network** (FFN), **residual connections** and **layer normalization**. The FFN is applied to each token independently and usually expands the dimension 4× ($d_{ff} = 4 \cdot d_{model}$) before projecting back:

$$\mathrm{FFN}(x) = W_2\,\mathrm{GELU}(W_1 x + b_1) + b_2$$

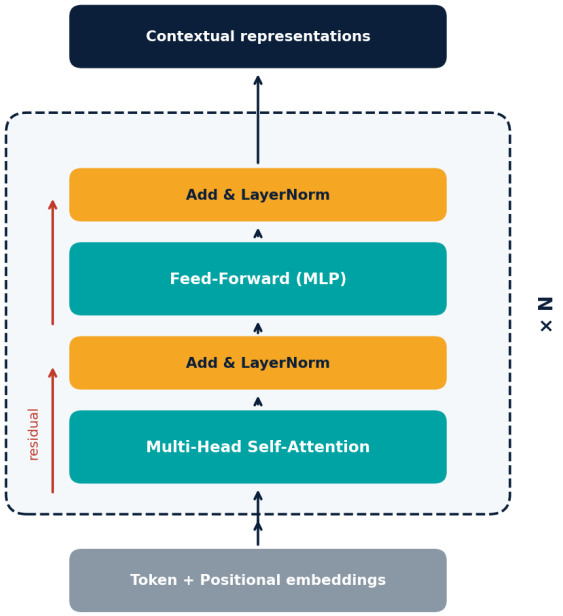


Figure 4: One Transformer layer (stacked N times).

- **Residual connections** (x + Sublayer(x)) let gradients bypass the sublayer, enabling very deep stacks.
- **LayerNorm** normalizes each token's features (not across the batch), stabilizing training; modern models often place it *before* each sublayer ("pre-norm").
- **Parallelism:** all positions are processed at once during training, which is why Transformers train far faster on GPUs than RNNs.

4 Multi-Head Attention

One attention head produces a single weighted average, so it can focus on only one kind of relationship at a time. **Multi-head attention** runs **h** attention operations in parallel, each in its own lower-dimensional subspace, then concatenates the results. Different heads learn to capture different patterns, for example syntactic dependencies, coreference ("it" → "animal"), neighbouring-token patterns or rare-word focus.

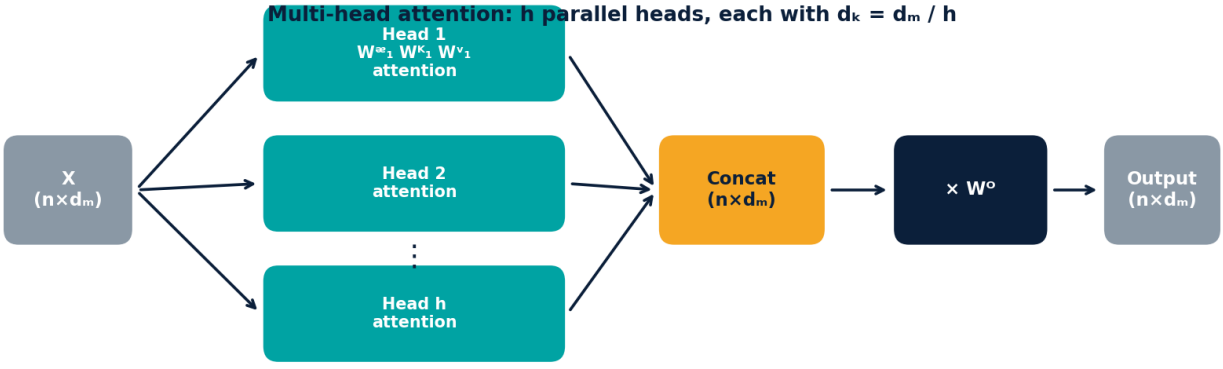


Figure 5: Multi-head attention: split into heads, attend in parallel, concatenate, project.

$$\text{head}_i = \text{Attention}\left(XW_Q^{(i)},\, XW_K^{(i)},\, XW_V^{(i)}\right), \qquad \text{MHA}(X) = \text{Concat}(\text{head}_1,\, \ldots,\, \text{head}_h)\, W_O$$

The head dimension is $d_k = d_{model}/h$, so the **total computation stays about the same** as one big head while the model gains several “views”.

Quantity	Formula	BERT-base (d_{model} = 768, h = 12)
Head size d_k	d_{model} / h	768 / 12 = **64**
Attention parameters	$4 \cdot d_{model}^2$ (W_Q, W_K, W_V, W_O), independent of h	4×768^2 = **2,359,296** (+ biases)
FFN parameters	$2 \cdot d_{model} \cdot d_{ff}$, with $d_{ff} = 4 \cdot d_{model}$	2 × 768 × 3072 = **4,718,592** (+ biases)
Score matrices per layer	h matrices of size n × n	12 matrices; n = 512 gives 12 × 262,144 scores

WORKED EXAMPLE
BERT-base has 12 layers. Each layer ≈ 2.36M (attention) + 4.72M (FFN) ≈ 7.1M parameters, so 12 layers ≈ 85M. Adding ≈ 23.4M token embeddings, position and segment embeddings and the pooler gives the commonly quoted ≈ **110M** parameters.

- Using more heads does not add parameters in the attention projections (each head just gets a smaller d_k), but too many heads make each one too small to be useful.
- Some heads can be pruned with little loss in accuracy, showing redundancy.
- **Efficiency variants:** multi-query attention (MQA) and grouped-query attention (GQA) share keys/values across heads to shrink the inference-time **KV cache**.

5 BERT: Bidirectional Encoder Representations from Transfo

BERT (Google, 2018) is an **encoder-only** Transformer. Because it has no causal mask, every token attends to both its left and right context, producing deep **bidirectional** representations. It is pre-trained on large unlabelled text and then **fine-tuned** for specific tasks.

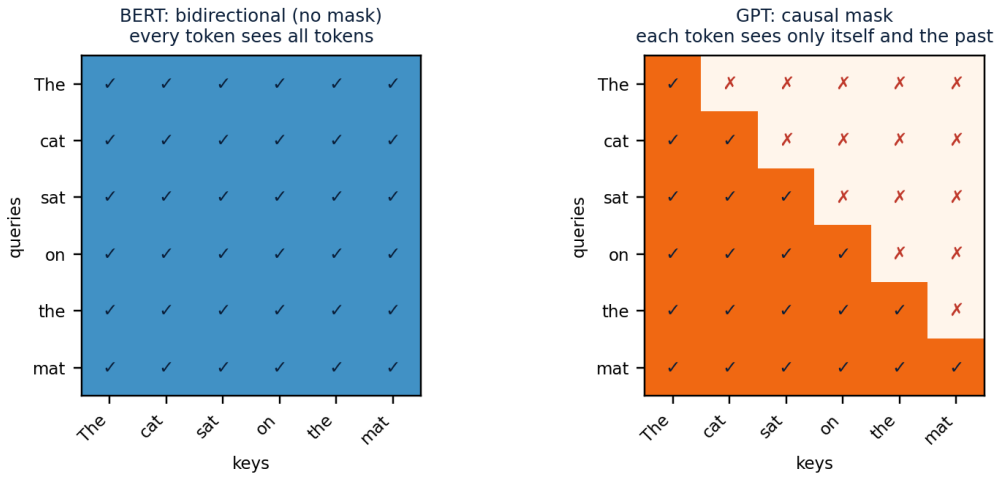


Figure 6: BERT (left) lets tokens see the whole sequence; GPT (right) hides the future. ✓ = attention allowed.

5.1 Input representation

Input = **[CLS]** sentence A **[SEP]** sentence B **[SEP]**. Each token’s input vector is the **sum of three embeddings**: **token** (WordPiece) + **segment** (A or B) + **position** (learned, up to 512).

5.2 Pre-training objectives

Masked Language Modelling (MLM): randomly choose 15% of tokens and predict them from the surrounding context. Of the chosen tokens, 80% are replaced by [MASK], 10% by a random token and 10% are left unchanged, so the model cannot rely on seeing [MASK] at fine-tuning time (it never appears there).

$$L_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P(x_i \mid x_{\setminus \mathcal{M}})$$

WORKED EXAMPLE
"The cat [MASK] on the mat." BERT uses both "The cat" (left) and "on the mat" (right) to predict **sat**. A left-to-right model could only use "The cat".

Next Sentence Prediction (NSP): given sentences A and B, predict whether B truly follows A (50% random). Later work (RoBERTa) found NSP unnecessary and dropped it, gaining accuracy from more data and longer training.

5.3 Sizes and fine-tuning

Model	Layers	Hidden size	Heads	Parameters
BERT-base	12	768	12	≈ 110M
BERT-large	24	1024	16	≈ 340M

Task	Head added on top of BERT	Uses
Sentence / document classification (sentiment, intent)	Linear layer on the **[CLS]** vector	Sequence-level output
Token classification (NER, POS tagging)	Linear layer on **every token** vector	Token-level outputs
Extractive question answering	Two linear layers predicting **start** and **end** positions of the answer span	Span prediction
Sentence-pair tasks (entailment, similarity)	[CLS] over "A [SEP] B"	Cross-encoding of both sentences

- **Strengths:** excellent for *understanding* tasks (classification, retrieval, tagging); fine-tunes with small labelled datasets; often small enough to run cheaply.
- **Limitations:** not designed to generate text; fixed maximum length (512); pre-train–fine-tune mismatch from [MASK]; learns from only ~15% of tokens per pass.
- **Family:** RoBERTa (better training recipe), ALBERT (parameter sharing), **DistilBERT** (distilled: ~40% smaller, ~60% faster, retains ~97% of BERT's language-understanding performance), DeBERTa, multilingual BERT.

6 GPT: Generative Pre-trained Transformer

GPT (OpenAI, 2018) is a **decoder-only** Transformer trained as an **autoregressive language model**: predict the next token given all previous ones. A **causal mask** (Figure 6, right) ensures position t cannot see positions after t, so the same model can generate text one token at a time.

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t}), \qquad L = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})$$

Training is **self-supervised**: the "label" for each position is simply the next token of the raw text, so no human annotation is needed and every token provides a learning signal.

6.1 Generation and decoding

At inference the model outputs a probability distribution over the vocabulary; one token is chosen, appended to the input, and the process repeats until an end token or length limit. A **KV cache** stores keys and values of earlier tokens so each new token needs only incremental computation.

Strategy	How it picks the next token	Effect
Greedy	Highest-probability token	Deterministic, but repetitive and can miss better sequences
Beam search	Keeps the k best partial sequences	Good for translation / summarization; can be bland
Temperature T	Samples from softmax(z / T)	T < 1 sharper/safer, T > 1 flatter/more random
Top-k	Sample only among the k most likely tokens	Cuts off the unlikely tail
Top-p (nucleus)	Sample from the smallest set whose cumulative probability ≥ p	Adapts the candidate set to the model's confidence

WORKED EXAMPLE
Logits z = [2, 1, 0]. T = 1: probabilities = **[0.665, 0.245, 0.090]**. T = 0.5 (z/T = [4, 2, 0]): **[0.867, 0.117, 0.016]**, sharper. T = 2 (z/T = [1, 0.5, 0]): **[0.506, 0.307, 0.186]**, flatter. As T → 0 sampling approaches greedy decoding.

6.2 Scale and in-context learning

Model	Year	Parameters	Notable idea
GPT-1	2018	≈ 117M	Generative pre-training, then supervised fine-tuning
GPT-2	2019	up to 1.5B	Zero-shot task performance from pure language modelling
GPT-3	2020	175B	**In-context (few-shot) learning**: tasks specified through the prompt, with no weight updates

- **Scaling laws:** loss decreases predictably as model size, data and compute grow, motivating ever larger models trained on more tokens.
- **Alignment:** raw pre-trained models just continue text. **Supervised fine-tuning** on instruction–response pairs, then **RLHF** (reinforcement learning from human feedback: train a reward model from human preference rankings, then optimize the policy, e.g. with PPO) produce assistants that follow instructions (InstructGPT → ChatGPT-style models).
- **Prompting techniques:** zero-shot, few-shot examples, chain-of-thought ("think step by step"), and retrieval-augmented generation (RAG) to ground answers in documents.
- **Limitations: hallucination** (fluent but false output), sensitivity to prompt wording, finite context window, training-data bias and a knowledge cutoff.

7 Revision Kit

7.1 BERT vs GPT vs T5

	BERT	GPT	T5 / BART
Architecture	Encoder-only	Decoder-only	Encoder–decoder
Attention	Bidirectional (full)	Causal (left-to-right)	Full in encoder; causal + cross in decoder
Pre-training	Masked LM (+ NSP)	Next-token prediction	Span corruption / denoising (text-to-text)
Best for	Classification, NER, QA, embeddings	Open-ended generation, chat, code	Translation, summarization, text-to-text
Generates text?	Not natively	**Yes**	Yes

7.2 One-page formula sheet

Concept	Formula / rule
Embedding table size	$V \times d$ parameters
Cosine similarity	$u \cdot v \,/\, (\|u\|\,\|v\|)$
Sinusoidal PE	$\sin(pos/10000^{2i/d})$, $\cos(pos/10000^{2i/d})$
Scaled dot-product attention	$\text{softmax}(QK^{\top}/\sqrt{d_k})\,V$
Head size	$d_k = d_{model} / h$
Attention params (per layer)	$4 \cdot d_{model}^{2}$ (+ biases)
FFN params (per layer)	$2 \cdot d_{model} \cdot d_{ff}$, $d_{ff} = 4 \cdot d_{model}$
Attention cost	$O(n^2 \cdot d)$ time, $O(n^2)$ memory
MLM masking	15% selected: 80% [MASK], 10% random, 10% unchanged
Autoregressive loss	$-\Sigma \log P(x_t \mid x_{<t})$
Temperature	$\text{softmax}(z / T)$

7.3 Classic mistakes

- Treating tokens as words, or using a tokenizer different from the model's.
- Believing static embeddings (Word2Vec) give different vectors for different senses of a word.
- Forgetting positional information: attention alone is order-blind.
- Omitting the $\sqrt{d_k}$ scaling, or applying softmax over the wrong axis (it must normalize over keys, i.e. each row).
- Saying attention is $O(n)$ in sequence length (it is $O(n^2)$).
- Confusing BERT's bidirectional MLM with GPT's causal next-token prediction.
- Expecting BERT to generate long text, or GPT to be "bidirectional".
- Assuming more heads means more attention parameters.
- Believing a larger context window removes the need to think about the quadratic cost.
- Trusting fluent output as factual (hallucination).

7.4 Self-check before the MCQ assessment

- Can you run three BPE merges on a small corpus?

- Can you compute a two-token attention output by hand, including the softmax?
- Can you explain why dividing by $\sqrt{d_k}$ matters and what a causal mask does?
- Can you count parameters of an embedding table and of one Transformer layer?
- Can you choose BERT, GPT or T5 for a given task and justify it?

KEY IDEA

Next step: attempt the **Module 03 MCQ Assessment**. For each miss, return to the matching section, redo the worked example by hand and write one sentence on why the wrong option was tempting.